

Day 9 learning(20/03/2026)

#Serialization in Java:

1. What is Serialization?

Serialization is a mechanism in Java where an **Object's state is converted into a byte stream**. This byte stream can then be saved to a database, stored in a file, or sent over a network to another JVM. The reverse process, where the byte stream is used to recreate the actual Java object in memory, is called **Deserialization**.

2. How to Achieve Serialization

To make a class serializable, you must follow these steps:

- **Implement the Interface:** Your class must implement the `java.io.Serializable` interface. This is a **Marker Interface** (it has no methods; it just tells the JVM that this class can be serialized).
- **Use Object Streams:** Use `ObjectOutputStream` to write the object (Serialize) and `ObjectInputStream` to read the object (Deserialize).

3. The Role of serialVersionUID

The `serialVersionUID` is a unique identifier for each class used during the deserialization process to ensure that the sender and receiver of a serialized object have loaded classes for that object that are compatible.

Why is it declared as *private static final*?

- **Final:** To ensure the ID remains constant. If the structure of the class changes slightly, Java might calculate a different ID, leading to an `InvalidClassException`.
- **Static:** Because the ID belongs to the class itself, not to any individual object instance.
- **Private:** To prevent child classes or external classes from accessing or modifying it, as it is only relevant to the specific class it is defined in.

4. The *transient* Keyword

If you have sensitive data (like passwords) or fields that do not make sense to save (like temporary thread states), you can mark them with the `transient` keyword. **Transient variables are not serialized**; during deserialization, they return to their default values (e.g., null for objects, 0 for ints).

5. Serialization and Inheritance

- **Parent is Serializable:** If a parent class implements `Serializable`, the child class is automatically serializable.
- **Only Child is Serializable:** If only the child class implements the interface, the child's data will be saved, but the parent's data will not be preserved unless the parent has a no-arg constructor (it will reset to default values).

6. Object Graph

When you serialize an object, Java automatically tries to serialize all other objects referenced by it. This is called an **Object Graph**. If any object in this chain does not implement `Serializable`, the process will fail with a `NotSerializableException`.

7. Externalizable Interface

For developers who want total control over the serialization logic (to improve performance or add custom security), Java provides the `Externalizable` interface. Unlike `Serializable` (which is automatic), `Externalizable` requires you to manually implement two methods:

- `writeExternal()`
- `readExternal()`

8. Serialization in Singleton Pattern

Serialization can "break" a Singleton class because deserialization always creates a **new instance** of the object. To prevent this and maintain the Singleton property, you must implement a special method:

- `protected Object readResolve()`: This method is called during deserialization to return the existing instance instead of a new one.

#Synchronization in Java:

1. What is Synchronization?

Synchronization in Java is a capability to control the access of multiple threads to any shared resource. It is used to prevent **Thread Interference** and **Consistency Errors**. When a thread is executing a synchronized method or block, it acquires a **Lock (Monitor)** on that object. Other threads are "Blocked" from entering until the lock is released.

2. Types of Synchronization

- **Synchronized Method:** Locks the entire method for the current object (`this`).
 - Example: `public synchronized void deposit(int amount) { ... }`
- **Synchronized Block:** Locks only a specific part of the code. This is more efficient than a synchronized method because it reduces the scope of the lock.
 - Example: `synchronized(this) { balance += amount; }`
- **Static Synchronization:** If the method is static, the lock is on the **Class** rather than the instance.

3. The Volatile Keyword

The `volatile` keyword is used to mark a Java variable as "being stored in main memory." It ensures **Visibility** of changes to variables across threads.

How it Works:

- Normally, threads cache variables in local CPU registers for performance.
- With `volatile`, every read of a volatile variable will be from the computer's **Main Memory**, and every write to a volatile variable will be written to main memory.

4. Deadlock in Synchronization

A **Deadlock** is a situation where two or more threads are blocked forever, waiting for each other to release locks.

- *Example:* Thread 1 has Lock A and waits for Lock B. Thread 2 has Lock B and waits for Lock A.

5. Best Practices

- Use **Synchronized Blocks** over Synchronized Methods to improve performance.
- Use `volatile` only when you need **Visibility** and the operation is simple (like a boolean flag).
- For complex counter operations (like `count++`), use `java.util.concurrent.atomic` classes like `AtomicInteger` instead of just `volatile`.

4. Comparison: Volatile vs. Synchronized

Feature	Volatile	Synchronized
Type	Variable Modifier	Method/Block Modifier
Visibility	Guarantees Visibility	Guarantees Visibility
Atomicity	No (e.g., <code>i++</code> is not safe)	Yes (Atomic execution)
Locking	Non-blocking	Blocking
Performance	High	Low (due to overhead)

#Optional class in Java:

1. What is Optional Class?

`java.util.Optional<T>` ek container object hai jise Java 8 mein introduce kiya gaya tha. Iska main maqsad **NullPointerException (NPE)** ko handle karna aur code ko zyada readable banana hai.

Ye ek aisa box hai jo ya toh ek **Non-null value** hold karta hai ya fir **Khaali (Empty)** hota hai.

2. Why use Optional?

Pehle jab koi method `null` return karta tha, toh developer ko har jagah `if (obj != null)` check lagana padta tha. Agar koi check bhool gaya, toh program crash ho jata tha. `Optional` use karne se developer ko pata hota hai ki value missing ho sakti hai, isliye wo use handle karne ke liye majboor hota hai.

3. How to Create Optional Objects

Optional ke objects banane ke teen main tarike hain:

- **`Optional.of(value)`**: Agar aap pakka hain ki value null nahi hogi. Agar value null hui, toh ye turant `NullPointerException` de dega.
- **`Optional.ofNullable(value)`**: Agar value null ho bhi sakti hai aur nahi bhi. Ye sabse zyada use hone wala method hai.
- **`Optional.empty()`**: Ek khaali (Empty) Optional object banane ke liye.

4. Important Methods in Optional

- **`isPresent()`**: Return karta hai `true` agar value maujood hai, warna `false`.
- **`ifPresent(Consumer)`**: Agar value maujood hai, toh hi us par koi action perform karta hai.
- **`get()`**: Value nikaalne ke liye. (Lekin agar value null hui toh ye exception de dega, isliye ise check ke bina use na karein).
- **`orElse(defaultValue)`**: Agar value missing hai, toh ek default value return karta hai.
- **`orElseThrow()`**: Agar value missing hai, toh exception throw karta hai.

5. Functional Style Methods (Map, Filter, FlatMap)

Optional class functional programming ko support karti hai:

- **filter():** Agar value kisi condition ko match karti hai toh wahi value return hogi, warna empty Optional milega.
- **map():** Value ko transform karne ke liye (e.g., String ko UpperCase mein badalna).

6. When NOT to use Optional

- **Class Fields:** Optional ko class ke variables (fields) ke roop mein use nahi karna chahiye kyunki ye **Serializable** nahi hai.
- **Method Parameters:** Ise method ke arguments mein pass karna achhi practice nahi maani jati.
- **Collections:** **Optional<List<String>>** use karne se behtar hai ki aap ek empty list return karein.

7. Best Practices

1. Hamesha **Return Type** ke liye Optional ka use karein jahan value missing ho sakti hai.
2. **get()** method use karne se bachein, uski jagah **orElse()** ya **ifPresent()** ka use karein.
3. **Optional** ka use sirf null safety ke liye karein, ise har jagah replace na karein kyunki ye memory consume karta hai.

#Quick Comparison Table

Method	Behavior if Value is Null
Optional.of()	Throws NullPointerException
Optional.ofNullable()	Returns Empty Optional
orElse()	Returns Default Value
orElseGet()	Calls a Function to get Default Value

